This chapter provides an overview of the key concepts embodied in C++. C++ is an object-oriented programming language, and its object-oriented features are highly interrelated. In several instances, this interrelatedness makes it difficult to describe one feature of C++ without implicitly involving several others. Moreover, the object-oriented features of C++ are, in many places, so intertwined that discussion of one feature *implies* prior knowledge of another. To address this problem, this chapter presents a quick overview of the most important aspects of C++, including its history, its key features, and the difference between traditional and Standard C++. The remaining chapters examine C++ in detail.

The Origins of C++

C++ began as an expanded version of C. The C++ extensions were first invented by Bjarne Stroustrup in 1979 at Bell Laboratories in Murray Hill, New Jersey. He initially called the new language "C with Classes." However, in 1983 the name was changed to C++.

Although C was one of the most liked and widely used professional programming languages in the world, the invention of C++ was necessitated by one major programming factor: increasing complexity. Over the years, computer programs have become larger and more complex. Even though C is an excellent programming language, it has its limits. In C, once a program exceeds from 25,000 to 100,000 lines of code, it becomes so complex that it is difficult to grasp as a totality. The purpose of C++ is to allow this barrier to be broken. The essence of C++ is to allow the programmer to comprehend and manage larger, more complex programs.

Most additions made by Stroustrup to C support object-oriented programming, sometimes referred to as OOP. (See the next section for a brief explanation of object-oriented programming.) Stroustrup states that some of C++'s object-oriented features were inspired by another object-oriented language called Simula67. Therefore, C++ represents the blending of two powerful programming methods.

Since C++ was first invented, it has undergone three major revisions, with each adding to and altering the language. The first revision was in 1985 and the second in 1990. The third occurred during the standardization of C++. Several years ago, work began on a standard for C++. Toward that end, a joint ANSI (American National Standards Institute) and ISO (International Standards Organization) standardization committee was formed. The first draft of the proposed standard was created on January 25, 1994. In that draft, the ANSI/ISO C++ committee (of which I was a member) kept the features first defined by Stroustrup and added some new ones as well. But in general, this initial draft reflected the state of C++ at the time.

Soon after the completion of the first draft of the C++ standard, an event occurred that caused the language to be greatly expanded: the creation of the Standard Template Library (STL) by Alexander Stepanov. The STL is a set of generic routines that you can use to manipulate data. It is both powerful and elegant, but also quite large. Subsequent

to the first draft, the committee voted to include the STL in the specification for C++. The addition of the STL expanded the scope of C++ well beyond its original definition. While important, the inclusion of the STL, among other things, slowed the standardization of C++.

It is fair to say that the standardization of C++ took far longer than anyone had expected when it began. In the process, many new features were added to the language and many small changes were made. In fact, the version of C++ defined by the C++ committee is much larger and more complex than Stroustrup's original design. The final draft was passed out of committee on November 14, 1997 and an ANSI/ISO standard for C++ became a reality in 1998. This specification for C++ is commonly referred to as *Standard C++*.

The material in this book describes Standard C++, including all of its newest features. This is the version of C++ created by the ANSI/ISO standardization committee, and it is the one that is currently accepted by all major compilers.

What Is Object-Oriented Programming?

Since object-oriented programming (OOP) drove the creation of C++, it is necessary to understand its foundational principles. OOP is a powerful way to approach the job of programming. Programming methodologies have changed dramatically since the invention of the computer, primarily to accommodate the increasing complexity of programs. For example, when computers were first invented, programming was done by toggling in the binary machine instructions using the computer's front panel. As long as programs were just a few hundred instructions long, this approach worked. As programs grew, assembly language was invented so that a programmer could deal with larger, increasingly complex programs, using symbolic representations of the machine instructions. As programs continued to grow, high-level languages were introduced that gave the programmer more tools with which to handle complexity. The first widespread language was, of course, FORTRAN. Although FORTRAN was a very impressive first step, it is hardly a language that encourages clear, easy-to-understand programs.

The 1960s gave birth to structured programming. This is the method encouraged by languages such as C and Pascal. The use of structured languages made it possible to write moderately complex programs fairly easily. Structured languages are characterized by their support for stand-alone subroutines, local variables, rich control constructs, and their lack of reliance upon the GOTO. Although structured languages are a powerful tool, they reach their limit when a project becomes too large.

Consider this: At each milestone in the development of programming, techniques and tools were created to allow the programmer to deal with increasingly greater complexity. Each step of the way, the new approach took the best elements of the previous methods and moved forward. Prior to the invention of OOP, many projects were nearing (or exceeding) the point where the structured approach no longer

worked. Object-oriented methods were created to help programmers break through these barriers.

Object-oriented programming took the best ideas of structured programming and combined them with several new concepts. The result was a different way of organizing a program. In the most general sense, a program can be organized in one of two ways: around its code (what is happening) or around its data (who is being affected). Using only structured programming techniques, programs are typically organized around code. This approach can be thought of as "code acting on data." For example, a program written in a structured language such as C is defined by its functions, any of which may operate on any type of data used by the program.

Object-oriented programs work the other way around. They are organized around data, with the key principle being "data controlling access to code." In an object-oriented language, you define the data and the routines that are permitted to act on that data. Thus, a data type defines precisely what sort of operations can be applied to that data.

To support the principles of object-oriented programming, all OOP languages have three traits in common: encapsulation, polymorphism, and inheritance. Let's examine each.

Encapsulation

Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse. In an object-oriented language, code and data may be combined in such a way that a self-contained "black box" is created. When code and data are linked together in this fashion, an *object* is created. In other words, an object is the device that supports encapsulation.

Within an object, code, data, or both may be *private* to that object or *public*. Private code or data is known to and accessible only by another part of the object. That is, private code or data may not be accessed by a piece of the program that exists outside the object. When code or data is public, other parts of your program may access it even though it is defined within an object. Typically, the public parts of an object are used to provide a controlled interface to the private elements of the object.

For all intents and purposes, an object is a variable of a user-defined type. It may seem strange that an object that links both code and data can be thought of as a variable. However, in object-oriented programming, this is precisely the case. Each time you define a new type of object, you are creating a new data type. Each specific instance of this data type is a compound variable.

Polymorphism

Object-oriented programming languages support *polymorphism*, which is characterized by the phrase "one interface, multiple methods." In simple terms, polymorphism is the attribute that allows one interface to control access to a general class of actions. The

specific action selected is determined by the exact nature of the situation. A real-world example of polymorphism is a thermostat. No matter what type of furnace your house has (gas, oil, electric, etc.), the thermostat works the same way. In this case, the thermostat (which is the interface) is the same no matter what type of furnace (method) you have. For example, if you want a 70-degree temperature, you set the thermostat to 70 degrees. It doesn't matter what type of furnace actually provides the heat.

This same principle can also apply to programming. For example, you might have a program that defines three different types of stacks. One stack is used for integer values, one for character values, and one for floating-point values. Because of polymorphism, you can define one set of names, **push( )** and **pop( )**, that can be used for all three stacks. In your program you will create three specific versions of these functions, one for each type of stack, but names of the functions will be the same. The compiler will automatically select the right function based upon the data being stored. Thus, the interface to a stack—the functions **push( )** and **pop( )**—are the same no matter which type of stack is being used. The individual versions of these functions define the specific implementations (methods) for each type of data.

Polymorphism helps reduce complexity by allowing the same interface to be used to access a general class of actions. It is the compiler's job to select the *specific action* (i.e., method) as it applies to each situation. You, the programmer, don't need to do this selection manually. You need only remember and utilize the *general interface*.

The first object-oriented programming languages were interpreters, so polymorphism was, of course, supported at run time. However, C++ is a compiled language. Therefore, in C++, both run-time and compile-time polymorphism are supported.

Inheritance

Inheritance is the process by which one object can acquire the properties of another object. This is important because it supports the concept of *classification*. If you think about it, most knowledge is made manageable by hierarchical classifications. For example, a Red Delicious apple is part of the classification *apple*, which in turn is part of the *fruit* class, which is under the larger class *food*. Without the use of classifications, each object would have to define explicitly all of its characteristics. However, through the use of classifications, an object need only define those qualities that make it unique within its class. It is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case. As you will see, inheritance is an important aspect of object-oriented programming.

Some C++ Fundamentals

In Part One, the C subset of C++ was described and C programs were used to demonstrate those features. From this point forward, all examples will be "C++

programs." That is, they will be making use of features unique to C++. For ease of discussion, we will refer to these C++-specific features simply as "C++ features" from now on.

If you come from a C background, or if you have been studying the C subset programs in Part One, be aware that C++ programs differ from C programs in some important respects. Most of the differences have to do with taking advantage of C++'s object-oriented capabilities. But C++ programs differ from C programs in other ways, including how I/O is performed and what headers are included. Also, most C++ programs share a set of common traits that clearly identify them *as* C++ programs. Before moving on to C++'s object-oriented constructs, an understanding of the fundamental elements of a C++ program is required.

This section describes several issues relating to nearly all C++ programs. Along the way, some important differences with C and earlier versions of C++ are pointed out.

A Sample C++ Program

Let's start with the short sample C++ program shown here.

```
#include <iostream>
using namespace std;

int main()
{
  int i;

  cout << "This is output.\n";  // this is a single line comment
  /* you can still use C style comments */

  // input a number using >>
  cout << "Enter a number: ";
  cin >> i;

  // now, output a number using <<
  cout << i << " squared is " << i*i << "\n";

  return 0;
}
```

As you can see, this program looks much different from the C subset programs found in Part One. A line-by-line commentary will be useful. To begin, the header **<iostream>** is included. This header supports C++-style I/O operations. (**<iostream>** is to C++ what **stdio.h** is to C.) Notice one other thing: there is no **.h** extension to the

name **iostream**. The reason is that **<iostream>** is one of the modern-style headers defined by Standard C++. Modern C++ headers do not use the **.h** extension.

The next line in the program is

```
using namespace std;
```

This tells the compiler to use the **std** namespace. Namespaces are a recent addition to C++. A namespace creates a declarative region in which various program elements can be placed. Namespaces help in the organization of large programs. The **using** statement informs the compiler that you want to use the **std** namespace. This is the namespace in which the entire Standard C++ library is declared. By using the **std** namespace you simplify access to the standard library. The programs in Part One, which use only the C subset, don't need a namespace statement because the C library functions are also available in the default, global namespace.

Note *Since both new-style headers and namespaces are recent additions to C++, you may encounter older code that does not use them. Also, if you are using an older compiler, it may not support them. Instructions for using an older compiler are found later in this chapter.*

Now examine the following line.

```
int main()
```

Notice that the parameter list in **main( )** is empty. In C++, this indicates that **main( )** has no parameters. This differs from C. In C, a function that has no parameters must use **void** in its parameter list, as shown here:

```
int main(void)
```

This was the way **main( )** was declared in the programs in Part One. However, in C++, the use of **void** is redundant and unnecessary. As a general rule, in C++ when a function takes no parameters, its parameter list is simply empty; the use of **void** is not required.

The next line contains two C++ features.

```
cout << "This is output.\n";  // this is a single line comment
```

First, the statement

```
cout << "This is output.\n";
```

causes **This is output**. to be displayed on the screen, followed by a carriage return-linefeed combination. In C++, the << has an expanded role. It is still the left shift operator, but when it is used as shown in this example, it is also an *output operator*. The word **cout** is an identifier that is linked to the screen. (Actually, like C, C++ supports I/O redirection, but for the sake of discussion, assume that **cout** refers to the screen.) You can use **cout** and the **<<** to output any of the built-in data types, as well as strings of characters.

Note that you can still use **printf( )** or any other of C's I/O functions in a C++ program. However, most programmers feel that using **<<** is more in the spirit of C++. Further, while using **printf( )** to output a string is virtually equivalent to using **<<** in this case, the C++ I/O system can be expanded to perform operations on objects that you define (something that you cannot do using **printf( )**).

What follows the output expression is a C++ *single-line comment*. As mentioned in Chapter 10, C++ defines two types of comments. First, you may use a multiline comment, which works the same in C++ as in C. You can also define a single-line comment by using **//**; whatever follows such a comment is ignored by the compiler until the end of the line is reached. In general, C++ programmers use multiline comments when a longer comment is being created and use single-line comments when only a short remark is needed.

Next, the program prompts the user for a number. The number is read from the keyboard with this statement:

```
cin >> i;
```

In C++, the **>>** operator still retains its right shift meaning. However, when used as shown, it also is C++'s *input operator*. This statement causes **i** to be given a value read from the keyboard. The identifier **cin** refers to the standard input device, which is usually the keyboard. In general, you can use **cin >>** to input a variable of any of the basic data types plus strings.

Note *The line of code just described is not misprinted. Specifically, there is not supposed to be an & in front of the **i**. When inputting information using a C-based function like **scanf( )**, you have to explicitly pass a pointer to the variable that will receive the information. This means preceding the variable name with the "address of" operator, &. However, because of the way the >> operator is implemented in C++, you do not need (in fact, must not use) the &. The reason for this is explained in Chapter 13.*

Although it is not illustrated by the example, you are free to use any of the C-based input functions, such as **scanf( )**, instead of using **>>**. However, as with **cout**, most programmers feel that **cin >>** is more in the spirit of C++.

Another interesting line in the program is shown here:

```
cout << i << "squared is " << i*i << "\n";
```

Assuming that **i** has the value 10, this statement causes the phrase **10 squared is 100** to be displayed, followed by a carriage return-linefeed. As this line illustrates, you can run together several **<<** output operations.

The program ends with this statement:

```
return 0;
```

This causes zero to be returned to the calling process (which is usually the operating system). This works the same in C++ as it does in C. Returning zero indicates that the program terminated normally. Abnormal program termination should be signaled by returning a nonzero value. You may also use the values **EXIT_SUCCESS** and **EXIT_FAILURE** if you like.

A Closer Look at the I/O Operators

As stated, when used for I/O, the **<<** and **>>** operators are capable of handling any of C++'s built-in data types. For example, this program inputs a **float**, a **double**, and a string and then outputs them:

```
#include <iostream>
using namespace std;

int main()
{
  float f;
  char str[80];
  double d;

  cout << "Enter two floating point numbers: ";
  cin >> f >> d;

  cout << "Enter a string: ";
  cin >> str;

  cout << f << " " << d << " " << str;

  return 0;
}
```

When you run this program, try entering **This is a test.** when prompted for the string. When the program redisplays the information you entered, only the word "This" will be displayed. The rest of the string is not shown because the **>>** operator stops reading input when the first white-space character is encountered. Thus, "is a test" is

never read by the program. This program also illustrates that you can string together several input operations in a single statement.

The C++ I/O operators recognize the entire set of backslash character constants described in Chapter 2. For example, it is perfectly acceptable to write

```
cout << "A\tB\tC";
```

This statement outputs the letters A, B, and C, separated by tabs.

Declaring Local Variables

If you come from a C background, you need to be aware of an important difference between C and C++ regarding when local variables can be declared. In C89, you must declare all local variables used within a block at the start of that block. You cannot declare a variable in a block after an "action" statement has occurred. For example, in C89, this fragment is incorrect:

```
/* Incorrect in C89. OK in C++. */
int f()
{
  int i;
  i = 10;

  int j;  /* won't compile as a C program */
  j = i*2;

  return j;
}
```

In a C89 program, this function is in error because the assignment intervenes between the declaration of **i** and that of **j**. However, when compiling it as a C++ program, this fragment is perfectly acceptable. In C++ (and C99) you may declare local variables at any point within a block—not just at the beginning.

Here is another example. This version of the program from the preceding section declares **str** just before it is needed.

```
#include <iostream>
using namespace std;

int main()
{
  float f;
```

```
  double d;
  cout << "Enter two floating point numbers: ";
  cin >> f >> d;

  cout << "Enter a string: ";
  char str[80];  // str declared here, just before 1st use
  cin >> str;

  cout << f << " " << d << " " << str;

  return 0;
}
```


Whether you declare all variables at the start of a block or at the point of first use is completely up to you. Since much of the philosophy behind C++ is the encapsulation of code and data, it makes sense that you can declare variables close to where they are used instead of just at the beginning of the block. In the preceding example, the declarations are separated simply for illustration, but it is easy to imagine more complex examples in which this feature of C++ is more valuable.

Declaring variables close to where they are used can help you avoid accidental side effects. However, the greatest benefit of declaring variables at the point of first use is gained in large functions. Frankly, in short functions (like many of the examples in this book), there is little reason not to simply declare variables at the start of a function. For this reason, this book will declare variables at the point of first use only when it seems warranted by the size or complexity of a function.

There is some debate as to the general wisdom of localizing the declaration of variables. Opponents suggest that sprinkling declarations throughout a block makes it harder, not easier, for someone reading the code to find quickly the declarations of all variables used in that block, making the program harder to maintain. For this reason, some C++ programmers do not make significant use of this feature. This book will not take a stand either way on this issue. However, when applied properly, especially in large functions, declaring variables at the point of their first use can help you create bug-free programs more easily.

No Default to int

A few years ago, there was a change to C++ that may affect older C++ code as well as C code being ported to C++. Both C89 and the original specification for C++ state that when no explicit type is specified in a declaration, type **int** is assumed. However, the "default-to-int" rule was dropped from C++ during standardization. C99 also drops this rule. However, there is still a large body of C and older C++ code that uses this rule.

The most common use of the "default-to-int" rule is with function return types. It was common practice to not specify **int** explicitly when a function returned an integer result. For example, in C89 and older C++ code the following function is valid.

```
func(int i)
{
  return i*i;
}
```

In Standard C++, this function must have the return type of **int** specified, as shown here.

```
int func(int i)
{
  return i*i;
}
```

As a practical matter, nearly all C++ compilers still support the "default-to-int" rule for compatibility with older code. However, you should not use this feature for new code because it is no longer allowed.

The bool Data Type

C++ defines a built-in Boolean type called **bool**. Objects of type **bool** can store only the values **true** or **false**, which are keywords defined by C++. As explained in Part One, automatic conversions take place which allow **bool** values to be converted to integers, and vice versa. Specifically, any non-zero value is converted to **true** and zero is converted to **false**. The reverse also occurs; **true** is converted to 1 and **false** is converted to zero. Thus, the fundamental concept of zero being false and non-zero being true is still fully entrenched in the C++ language.

Note *Although C89 (the C subset of C++) does not define a Boolean type, C99 adds to the C language a type called **_Bool**, which is capable of storing the values 1 and 0 (i.e., true/false). Unlike C++, C99 does not define **true** and **false** as keywords. Thus, **_Bool** as defined by C99 is incompatible with **bool** as defined by C++.*

*The reason that C99 specifies **_Bool** rather than **bool** as a keyword is that many preexisting C programs have aleady defined their own custom versions of **bool**. By defining the Boolean type as **_Bool**, C99 avoids breaking this preexisting code. However, it is possible to achieve compatibility between C++ and C99 on this point because C99 adds the header **<stdbool.h>** which defines the macros **bool**, **true**, and **false**. By including this header, you can create code that is compatible with both C99 and C++.*

The New C++ Headers

As you know, when you use a library function in a program, you must include its header. This is done using the **#include** statement. For example, in C, to include the header for the I/O functions, you include **stdio.h** with a statement like this:

```
#include <stdio.h>
```

Here, **stdio.h** is the name of the file used by the I/O functions, and the preceding statement causes that file to be included in your program. The key point is that this **#include** statement normally *includes a file*.

When C++ was first invented and for several years after that, it used the same style of headers as did C. That is, it used *header files*. In fact, Standard C++ still supports C-style headers for header files that you create and for backward compatibility. However, Standard C++ created a new kind of header that is used by the Standard C++ library. The new-style headers *do not* specify filenames. Instead, they simply specify standard identifiers that may be mapped to files by the compiler, although they need not be. The new-style C++ headers are an abstraction that simply guarantee that the appropriate prototypes and definitions required by the C++ library have been declared.

Since the new-style headers are not filenames, they do not have a **.h** extension. They consist solely of the header name contained between angle brackets. For example, here are some of the new-style headers supported by Standard C++.

<iostream> <fstream> <vector> <string>

The new-style headers are included using the **#include** statement. The only difference is that the new-style headers do not necessarily represent filenames.

Because C++ includes the entire C function library, it still supports the standard C-style header files associated with that library. That is, header files such as **stdio.h** or **ctype.h** are still available. However, Standard C++ also defines new-style headers that you can use in place of these header files. The C++ versions of the C standard headers simply add a "c" prefix to the filename and drop the **.h**. For example, the C++ new-style header for **math.h** is **<cmath>**. The one for **string.h** is **<cstring>**. Although it is currently permissible to include a C-style header file when using C library functions, this approach is deprecated by Standard C++ (that is, it is not recommended). For this reason, from this point forward, this book will use new-style C++ headers in all **#include** statements. If your compiler does not support new-style headers for the C function library, then simply substitute the old-style, C-like headers.

Since the new-style header is a relatively recent addition to C++, you will still find many, many older programs that don't use it. These programs employ C-style headers, in which a filename is specified. As the old-style skeletal program shows, the traditional way to include the I/O header is as shown here.

```
#include <iostream.h>
```

This causes the file **iostream.h** to be included in your program. In general, an old-style header file will use the same name as its corresponding new-style header with a **.h** appended.

As of this writing, all C++ compilers support the old-style headers. However, the old-style headers have been declared obsolete and their use in new programs is not recommended. This is why they are not used in this book.

Remember *While still common in existing C++ code, old-style headers are obsolete.*

Namespaces

When you include a new-style header in your program, the contents of that header are contained in the **std** namespace. A *namespace* is simply a declarative region. The purpose of a namespace is to localize the names of identifiers to avoid name collisions. Elements declared in one namespace are separate from elements declared in another. Originally, the names of the C++ library functions, etc., were simply put into the global namespace (as they are in C). However, with the advent of the new-style headers, the contents of these headers were placed in the **std** namespace. We will look closely at namespaces later in this book. For now, you won't need to worry about them because the statement

```
using namespace std;
```

brings the **std** namespace into visibility (i.e., it puts **std** into the global namespace). After this statement has been compiled, there is no difference between working with an old-style header and a new-style one.

One other point: for the sake of compatibility, when a C++ program includes a C header, such as **stdio.h**, its contents are put into the global namespace. This allows a C++ compiler to compile C-subset programs.

Working with an Old Compiler

As explained, both namespaces and the new-style headers are fairly recent additions to the C++ language, added during standardization. While all new C++ compilers support these features, older compilers may not. When this is the case, your compiler will report one or more errors when it tries to compile the first two lines of the sample programs in this book. If this is the case, there is an easy work-around: simply use an old-style header and delete the **namespace** statement. That is, just replace

```
#include <iostream>
using namespace std;
```

with

```
#include <iostream.h>
```

This change transforms a modern program into an old-style one. Since the old-style header reads all of its contents into the global namespace, there is no need for a **namespace** statement.

One other point: for now and for the next few years, you will see many C++ programs that use the old-style headers and do not include a **using** statement. Your C++ compiler will be able to compile them just fine. However, for new programs, you should use the modern style because it is the only style of program that complies with the C++ Standard. While old-style programs will continue to be supported for many years, they are technically noncompliant.

Introducing C++ Classes

This section introduces C++'s most important feature: the class. In C++, to create an object, you first must define its general form by using the keyword **class**. A **class** is similar syntactically to a structure. Here is an example. The following class defines a type called **stack**, which will be used to create a stack:

```
#define SIZE 100
```

```
// This creates the class stack.
class stack {
  int stck[SIZE];
  int tos;
public:
  void init();
  void push(int i);
  int pop();
};
```

A **class** may contain private as well as public parts. By default, all items defined in a **class** are private. For example, the variables **stck** and **tos** are private. This means that they cannot be accessed by any function that is not a member of the **class**. This is one way that encapsulation is achieved—access to certain items of data may be tightly controlled by keeping them private. Although it is not shown in this example, you can also define private functions, which then may be called only by other members of the **class**.

To make parts of a **class** public (that is, accessible to other parts of your program), you must declare them after the **public** keyword. All variables or functions defined after **public** can be accessed by all other functions in the program. Essentially, the rest of your program accesses an object through its public functions. Although you can have public variables, good practice dictates that you should try to limit their use. Instead, you should make all data private and control access to it through public functions. One other point: Notice that the **public** keyword is followed by a colon.

The functions **init( )**, **push( )**, and **pop( )** are called *member functions* because they are part of the class **stack**. The variables **stck** and **tos** are called *member variables* (or *data members*). Remember, an object forms a bond between code and data. Only member functions have access to the private members of their class. Thus, only **init( )**, **push( )**, and **pop( )** may access **stck** and **tos**.

Once you have defined a **class**, you can create an object of that type by using the class name. In essence, the class name becomes a new data type specifier. For example, this creates an object called **mystack** of type **stack**:

```
stack mystack;
```

When you declare an object of a class, you are creating an *instance* of that class. In this case, **mystack** is an instance of **stack**. You may also create objects when the **class** is defined by putting their names after the closing curly brace, in exactly the same way as you would with a structure.

To review: In C++, **class** creates a new data type that may be used to create objects of that type. Therefore, an object is an instance of a class in just the same way that some other variable is an instance of the **int** data type, for example. Put differently, a class is a

logical abstraction, while an object is real. (That is, an object exists inside the memory of the computer.)

The general form of a simple **class** declaration is

class *class-name* {
  private data and functions
public:
  public data and functions
} *object name list*;

Of course, the *object name list* may be empty.

Inside the declaration of **stack**, member functions were identified using their prototypes. In C++, all functions must be prototyped. Prototypes are not optional. The prototype for a member function within a class definition serves as that function's prototype in general.

When it comes time to actually code a function that is the member of a class, you must tell the compiler which class the function belongs to by qualifying its name with the name of the class of which it is a member. For example, here is one way to code the **push( )** function:

```
void stack::push(int i)
{
  if(tos==SIZE) {
    cout << "Stack is full.\n";
    return;
  }
  stck[tos] = i;
  tos++;
}
```

The **::** is called the *scope resolution operator*. Essentially, it tells the compiler that this version of **push( )** belongs to the **stack** class or, put differently, that this **push( )** is in **stack'**s scope. In C++, several different classes can use the same function name. The compiler knows which function belongs to which class because of the scope resolution operator.

When you refer to a member of a class from a piece of code that is not part of the class, you must always do so in conjunction with an object of that class. To do so, use the object's name, followed by the dot operator, followed by the name of the member. This rule applies whether you are accessing a data member or a function member. For example, this calls **init( )** for object **stack1**.

```
stack stack1, stack2;

stack1.init();
```

This fragment creates two objects, **stack1** and **stack2**, and initializes **stack1**. Understand that **stack1** and **stack2** are two separate objects. This means, for example, that initializing **stack1** does *not* cause **stack2** to be initialized as well. The only relationship **stack1** has with **stack2** is that they are objects of the same type.

Within a class, one member function can call another member function or refer to a data member directly, without using the dot operator. It is only when a member is referred to by code that does not belong to the class that the object name and the dot operator must be used.

The program shown here puts together all the pieces and missing details and illustrates the **stack** class:


```
#include <iostream>
using namespace std;

#define SIZE 100

// This creates the class stack.
class stack {
  int stck[SIZE];
  int tos;
public:
  void init();
  void push(int i);
  int pop();
};

void stack::init()
{
  tos = 0;
}

void stack::push(int i)
{
  if(tos==SIZE) {
    cout << "Stack is full.\n";
    return;
  }
  stck[tos] = i;
  tos++;
}

int stack::pop()
{
  if(tos==0) {
```

```
    cout << "Stack underflow.\n";
    return 0;
  }
  tos--;
  return stck[tos];
}

int main()
{
  stack stack1, stack2;  // create two stack objects

  stack1.init();
  stack2.init();

  stack1.push(1);
  stack2.push(2);

  stack1.push(3);
  stack2.push(4);

  cout << stack1.pop() << " ";
  cout << stack1.pop() << " ";
  cout << stack2.pop() << " ";
  cout << stack2.pop() << "\n";

  return 0;
}
```

The output from this program is shown here.

```
3 1 4 2
```

One last point: Recall that the private members of an object are accessible only by functions that are members of that object. For example, a statement like

```
stack1.tos = 0; // Error, tos is private.
```

could not be in the **main( )** function of the previous program because **tos** is private.

Constructors and Destructors

It is very common for some part of an object to require initialization before it can be used. For example, think back to the **stack** class developed earlier in this chapter. Before the stack could be used, **tos** had to be set to zero. This was performed by using the function **init( ).** Because the requirement for initialization is so common, C++ allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor function.

A *constructor* is a special function that is a member of a class and has the same name as that class. For example, here is how the **stack** class looks when converted to use a constructor for initialization:

```
// This creates the class stack.
class stack {
  int stck[SIZE];
  int tos;
public:
  stack();  // constructor
  void push(int i);
  int pop();
};
```

Notice that the constructor **stack( )** has no return type specified. In C++, constructors cannot return values and, thus, have no return type.

The **stack( )** constructor is coded like this:

```
// stack's constructor
stack::stack()
{
  tos = 0;
  cout << "Stack Initialized\n";
}
```

Keep in mind that the message **Stack Initialized** is output as a way to illustrate the constructor. In actual practice, most constructors will not output or input anything. They will simply perform various initializations.

An object's constructor is automatically called when the object is created. This means that it is called when the object's declaration is executed. If you are accustomed to thinking of a declaration statement as being passive, this is not the case for C++. In C++, a declaration statement is a statement that is executed. This distinction is not just academic. The code executed to construct an object may be quite significant. An object's constructor is called once for global or **static** local objects. For local objects, the constructor is called each time the object declaration is encountered.

The complement of the constructor is the *destructor*. In many circumstances, an object will need to perform some action or actions when it is destroyed. Local objects are created when their block is entered, and destroyed when the block is left. Global objects are destroyed when the program terminates. When an object is destroyed, its destructor (if it has one) is automatically called. There are many reasons why a destructor may be needed. For example, an object may need to deallocate memory that it had previously allocated or it may need to close a file that it had opened. In C++, it is the destructor that handles deactivation events. The destructor has the same name as the

constructor, but it is preceded by a ~. For example, here is the **stack** class and its constructor and destructor. (Keep in mind that the **stack** class does not require a destructor; the one shown here is just for illustration.)

```
// This creates the class stack.
class stack {
  int stck[SIZE];
  int tos;
public:
  stack();  // constructor
  ~stack(); // destructor
  void push(int i);
  int pop();
};

// stack's constructor
stack::stack()
{
  tos = 0;
  cout << "Stack Initialized\n";
}

// stack's destructor
stack::~stack()
{
  cout << "Stack Destroyed\n";
}
```

Notice that, like constructors, destructors do not have return values.

To see how constructors and destructors work, here is a new version of the **stack** program examined earlier in this chapter. Observe that **init( )** is no longer needed.

```
// Using a constructor and destructor.
#include <iostream>
using namespace std;

#define SIZE 100

// This creates the class stack.
class stack {
  int stck[SIZE];
  int tos;
public:
```

```
  stack();  // constructor
  ~stack(); // destructor
  void push(int i);
  int pop();
};

// stack's constructor
stack::stack()
{
  tos = 0;
  cout << "Stack Initialized\n";
}

// stack's destructor
stack::~stack()
{
  cout << "Stack Destroyed\n";
}

void stack::push(int i)
{
  if(tos==SIZE) {
    cout << "Stack is full.\n";
    return;
  }
  stck[tos] = i;
  tos++;
}

int stack::pop()
{
  if(tos==0) {
    cout << "Stack underflow.\n";
    return 0;
  }
  tos--;
  return stck[tos];
}

int main()
{
  stack a, b;  // create two stack objects
```

```
  a.push(1);
  b.push(2);

  a.push(3);
  b.push(4);

  cout << a.pop() << " ";
  cout << a.pop() << " ";
  cout << b.pop() << " ";
  cout << b.pop() << "\n";

  return 0;
}
```

This program displays the following:

```
Stack Initialized
Stack Initialized
3 1 4 2
Stack Destroyed
Stack Destroyed
```

The C++ Keywords

There are 63 keywords currently defined for Standard C++. These are shown in Table 11-1. Together with the formal C++ syntax, they form the C++ programming language. Also, early versions of C++ defined the **overload** keyword, but it is obsolete. Keep in mind that C++ is a case-sensitive language and it requires that all keywords be in lowercase.

asm	auto	bool	break
case	catch	char	class
const	const_cast	continue	default
delete	do	double	dynamic_cast
else	enum	explicit	export

Table 11-1. *The C++ keywords*

extern	false	float	for
friend	goto	if	inline
int	long	mutable	namespace
new	operator	private	protected
public	register	reinterpret_cast	return
short	signed	sizeof	static
static_cast	struct	switch	template
this	throw	true	try
typedef	typeid	typename	union
unsigned	using	virtual	void
volatile	wchar_t	while	

Table 11-1. *The C++ keywords* (continued)

The General Form of a C++ Program

Although individual styles will differ, most C++ programs will have this general form:

```
#includes
base-class declarations
derived class declarations
nonmember function prototypes
int main( )
{
    //...
}
nonmember function definitions
```

In most large projects, all **class** declarations will be put into a header file and included with each module. But the general organization of a program remains the same.

The remaining chapters in this section examine in greater detail the features discussed in this chapter, as well as all other aspects of C++.

Introduction to C++

- C++
  - Bjarne Stroustrup (Bell Labs, 1979)
  - started as extension to C (macros and variables)
  - added new useful, features
  - nowadays a language of its own
  - C++ (the next thing after C, though wouldn’t ++C be more appropriate?)

Outline

- Intro to C++
  - Object-Oriented Programming
  - Changes in C++
    - comments
    - variable declaration location
    - initialization
    - pointer changes
    - tagged structure type
    - enum types
    - bool type

Object-Oriented Programming

- First-class objects - atomic types in C
  - int, float, char
  - have:
    - values
    - sets of operations that can be applied to them
  - how represented irrelevant to how they are manipulated
- Other objects - structures in C
  - cannot be printed
  - do not have operations associated with them (at least, not directly)

Object-Oriented Idea

- Make all objects, whether C-defined or user-defined, first-class objects
- For C++ structures (called classes) allow:
  - functions to be associated with the class
  - only allow certain functions to access the internals of the class
  - allow the user to re-define existing functions (for example, input and output) to work on class

Classes of Objects in C++

- Classes
  - similar to structures in C (in fact, you can can still use the struct definition)
  - have fields corresponding to fields of a structure in C (similar to variables)
  - have fields corresponding to functions in C (functions that can be applied to that structure)
  - some fields are accessible by everyone, some not (data hiding)
  - some fields shared by the entire class

Instances of Classes in C++

- A class in C++ is like a type in C
- Variables created of a particular class are instances of that class
- Variables have values for fields of the class
- Class example: Student
  - has name, id, gpa, etc. fields that store values
  - has functions, changeGPA, addCredits, that can be applied to instances of that class
- Instance examples: John Doe, Jane Doe
  - each with their own values for the fields of the class

Comments in C++

- Can use C form of comments /* A Comment */
- Can also use // form:
  - when // encountered, remainder of line ignored
  - works only on that line
- Examples:

```
void main() {
   int I;  // Variable used in loops
   char C; // No comment comment
```

Variable Declarations

- In C++, variable declarations are not restricted to the beginnings of blocks (before any code)
  - you may interleave declarations/statements as needed
  - it is still good style to have declarations first
- Example

```
void main() {
  int I = 5;
  printf("Please enter J: ");
  int J; // Not declared at the start
  scanf("%d",&J);
```

Counter Variables in a For Loop

- You can declare the variable(s) used in a for loop in the initialization section of the for loop
  - good when counter used in for loop only exists in for loop (variable is throw-away)
- Example

```
for (int I = 0; I < 5; I++)
   printf("%d\n",I);
```

- Variable exists only during for loop (goes away when loop ends)

Initializing Global Variables

- Not restricted to using constant literal values in initializing global variables, can use any evaluable expression
- Example:

```
int rows = 5;
int cols = 6;
int size = rows * cols;

void main() {
...
```

Initializing Array Elements

- When giving a list of initial array values in C++, you can use expressions that have to be evaluated
- Values calculated at run-time before initialization done
- Example:

```
void main() {
  int n1, n2, n3;
  int *nptr[] = { &n1, &n2, &n3 };
```

void*

- In C it is legal to cast other pointers to and from a void *
- In C++ this is an error, to cast you should use an explicit casting command
- Example:

```
int N;
int *P = &N;
void *Q = P;            // illegal in C++
void *R = (void *) P; // ok
```

NULL in C++

- C++ does not use the value NULL, instead NULL is always 0 in C++, so we simply use 0
- Example:

```
int *P = 0; // equivalent to
            // setting P to NULL
```

- Can check for a 0 pointer as if true/false:

```
if (!P)  // P is 0 (NULL)
   ...
else // P is not 0 (non-NULL)
   ...
```

Tags and struct

- When using struct command in C++ (and for other tagged types), can create type using tag format and not use tag in variable declaration:

```
struct MyType {
  int A;
  float B;
};
MyType V;
```

enum in C++

- Enumerated types not directly represented as integers in C++
  - certain operations that are legal in C do not work in C++
- Example:

```
void main() {
   enum Color { red, blue, green };
   Color c = red;
   c = blue;
   c = 1;  // Error in C++
   ++c;  // Error in C++
```

bool

- C has no explicit type for true/false values
- C++ introduces type bool (later versions of C++)
  - also adds two new bool literal constants true (1) and false (0)
- Other integral types (int, char, etc.) are implicitly converted to bool when appropriate
  - non-zero values are converted to true
  - zero values are converted to false

bool operations

- Operators requiring bool value(s) and producing a bool value:

  && (And), || (Or), ! (Not)
- Relational operators (==, !=, <, >, <=, >=) produce bool values
- Some statements expect expressions that produce bool values:

```
if (boolean_expression)
 while (boolean_expression)
 do … while (boolean_expression)
 for ( ; boolean_expression; )
```

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#, etc.

The New C++ Headers

As you know, when you use a library function in a program, you must include its header. This is done using the **#include** statement. For example, in C, to include the header for the I/O functions, you include **stdio.h** with a statement like this:

```
#include <stdio.h>
```

Here, **stdio.h** is the name of the file used by the I/O functions, and the preceding statement causes that file to be included in your program. The key point is that this **#include** statement normally *includes a file*.

When C++ was first invented and for several years after that, it used the same style of headers as did C. That is, it used *header files*. In fact, Standard C++ still supports C-style headers for header files that you create and for backward compatibility. However, Standard C++ created a new kind of header that is used by the Standard C++ library. The new-style headers *do not* specify filenames. Instead, they simply specify standard identifiers that may be mapped to files by the compiler, although they need not be. The new-style C++ headers are an abstraction that simply guarantee that the appropriate prototypes and definitions required by the C++ library have been declared.

Since the new-style headers are not filenames, they do not have a **.h** extension. They consist solely of the header name contained between angle brackets. For example, here are some of the new-style headers supported by Standard C++.

<iostream> <fstream> <vector> <string>

The new-style headers are included using the **#include** statement. The only difference is that the new-style headers do not necessarily represent filenames.

Because C++ includes the entire C function library, it still supports the standard C-style header files associated with that library. That is, header files such as **stdio.h** or **ctype.h** are still available. However, Standard C++ also defines new-style headers that you can use in place of these header files. The C++ versions of the C standard headers simply add a "c" prefix to the filename and drop the **.h**. For example, the C++ new-style header for **math.h** is **<cmath>**. The one for **string.h** is **<cstring>**. Although it is currently permissible to include a C-style header file when using C library functions, this approach is deprecated by Standard C++ (that is, it is not recommended). For this reason, from this point forward, this book will use new-style C++ headers in all **#include** statements. If your compiler does not support new-style headers for the C function library, then simply substitute the old-style, C-like headers.

Since the new-style header is a relatively recent addition to C++, you will still find many, many older programs that don't use it. These programs employ C-style headers, in which a filename is specified. As the old-style skeletal program shows, the traditional way to include the I/O header is as shown here.

```
#include <iostream.h>
```

This causes the file **iostream.h** to be included in your program. In general, an old-style header file will use the same name as its corresponding new-style header with a **.h** appended.

As of this writing, all C++ compilers support the old-style headers. However, the old-style headers have been declared obsolete and their use in new programs is not recommended. This is why they are not used in this book.

Remember *While still common in existing C++ code, old-style headers are obsolete.*

Namespaces

When you include a new-style header in your program, the contents of that header are contained in the **std** namespace. A *namespace* is simply a declarative region. The purpose of a namespace is to localize the names of identifiers to avoid name collisions. Elements declared in one namespace are separate from elements declared in another. Originally, the names of the C++ library functions, etc., were simply put into the global namespace (as they are in C). However, with the advent of the new-style headers, the contents of these headers were placed in the **std** namespace. We will look closely at namespaces later in this book. For now, you won't need to worry about them because the statement

```
using namespace std;
```


brings the **std** namespace into visibility (i.e., it puts **std** into the global namespace). After this statement has been compiled, there is no difference between working with an old-style header and a new-style one.

One other point: for the sake of compatibility, when a C++ program includes a C header, such as **stdio.h**, its contents are put into the global namespace. This allows a C++ compiler to compile C-subset programs.

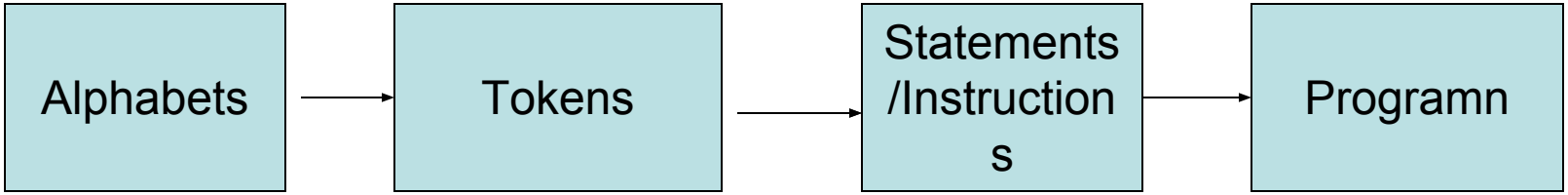
Unit – I: Introduction to C ++ Programming

Objectives

- Character set
- C++ Tokens
- Keywords
- Identifiers
- Constants
- Variables
- Data types
- Declaration of variables

Introduction

- C++ is High level programming language
- C++ language is used by programmers to interact with computer
- Steps to learn a programming language



Character set(Alphabets of C++)

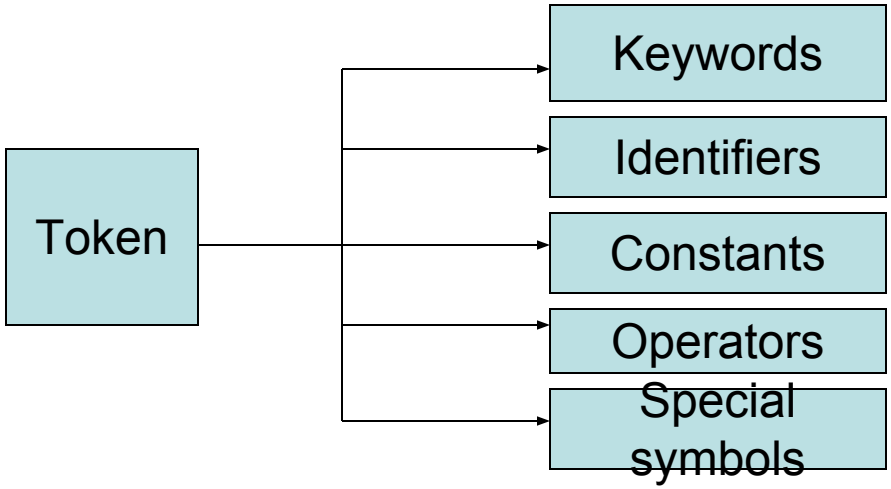
- Any symbol that is used while writing a C++ program is called character or an alphabet
- A Character can be a letter,digit or any special symbol all those charecters that can be used while writing C++ programs are called Character set.

Examples of Character set

- Letters like A to Z and a to z
- Digits like from 0 to 9
- Symbols such as # , ( ) { } [ ] % . ? “
- White spaces like \t ,\n

C++ Tokens

- A token is basic unit of any program
- One or more characters are grouped in sequence to form meaningful words.
- These meaningful words are called tokens



Keywords

- Certain words known as reserved for specific meaning in C++ language.
- Keywords should be writen in lower case letters.

Keywords

auto	default	float	register	struct	volatile
break	do	for	return	switch	while
case	double	goto	short	typedef	char
else	if	signed	union	const	enum
int	sizeof	unsigned	continue	extern	long
static	void				

Identifiers

- Identifiers refer to the names of variables, functions and arrays.
- These are user defined names and consist of a sequence of letters and digits, with a letter as a first character.
- The underscore character is also permitted in identifiers.

Identifiers

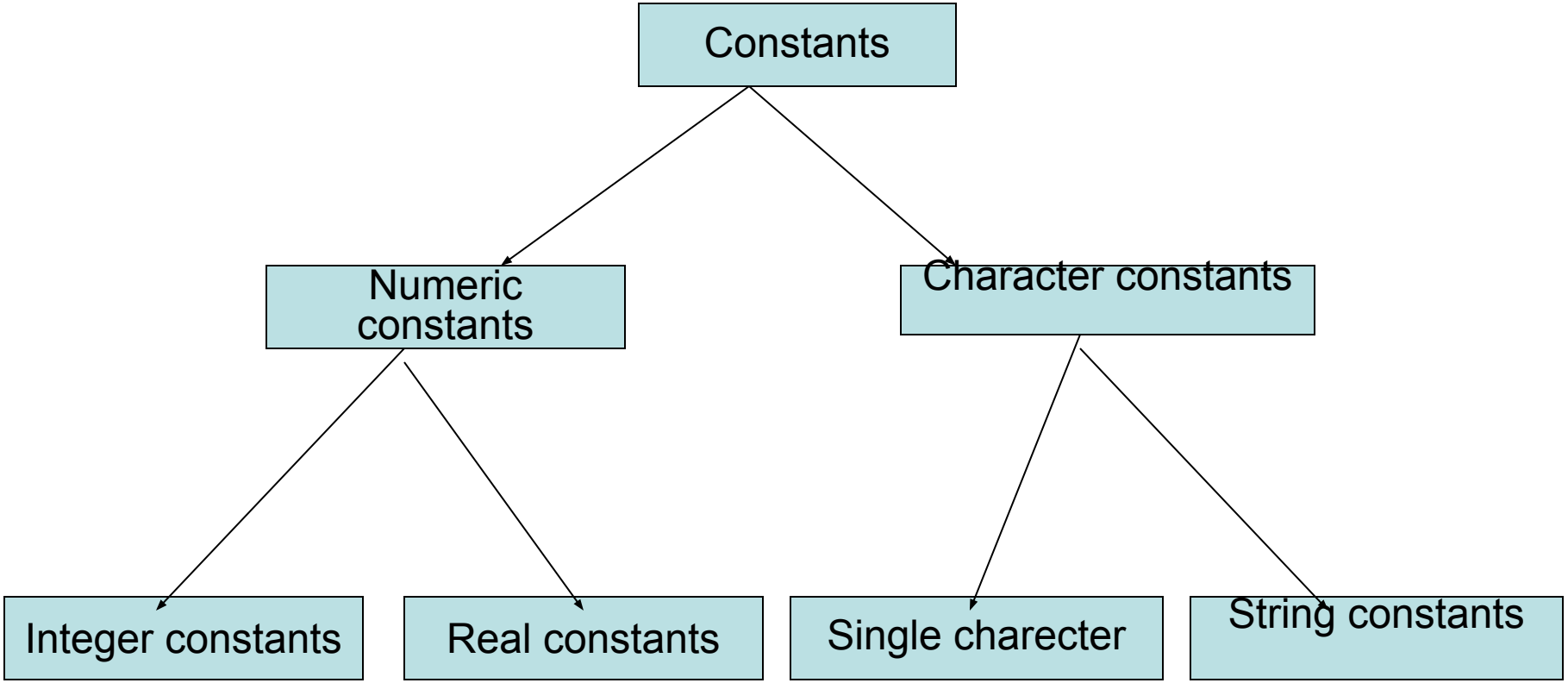
- **Rules of Identifiers:**

- **First character must be an alphabet(or underscore).**
- **Must consist of only letters, digits or underscore.**
- **Only first 31 characters are significant.**
- **Cannot use a keyword**
- **Must not contain white space.**

Constants

- Quantity which doesn’t change its value during the process of execution.
- C supports several types of constants.

Constants


Constants
Numeric constants
Character constants
Integer constants
Real constants
Single charecter
String constants

Rules of constructing integer constant

- Must have at least one digit
- Must not a decimal point
- It can be either positive r negative
- Default is positive
- No commas r blanks allowed within a constant
- 16 bit computer can handle integer numbers in the range of -32768 to 32767

Types of Integer constant

- Decimal – 10,140 ,-140
- Octal -- 010 ,0140,-0140
- Hexadecimal – 0x8A,0xAB

Definitions

- bit A single binary digit, that can have either value 0 or 1
- byte --8 bits.
- halfword --16 bits
- Word-- 32 bits
- Doubleword-- 64 bits

Floating point constants (real constants)

- Floating point constants also known as real constants
- whole number part and fraction part is separated by decimal point
- Two types of representation:
- 1. Fractional form representation.
- 2. Exponential form representation

Rules for constructing floating point constants (fractional form)

- Must have at least one digit
- It requires 4 bytes of memory
- Must have decimal point
- Can be either positive r negative
- Default sign is positive
- No commas r blanks allowed
- Examples +325.64 ,426.0,-32.76

Rules for constructing floating point constants (exponential form)

- Mantissa part and exponential part is separated by letter E.
- Mantissa can be either positive r negative
- Default sign of mantissa is positive
- Exponent must have at least one digit ,must be + or -, default is +
- No commas r blanks allowed
- Examples –4.1e8 ,
- 0.2e-5

Character constant

- It can have any valid single character from the character set enclosed between single quote.
- Each character is have a unique ascii value
- Ex ‘a’ , ‘=‘, ‘8’
- American Standard Code for Information Interchange (ASCII)
- Character constant requires 1 byte of memory for storage
- When character constant is stored in memory the corresponding ASCII value resides in the memory

Backslash character

- Format \(single character)
- It escapes the regular meaning of the character and gives some special meaning
- Ex: \n—new line character
- \’ –’
- This is otherwise called escape sequences

String constant

- Sequence of one r more characters enclosed between double quotes
- String always ends with ‘\0’
- “global”
- “\n”

Variables

- A variable is data name that may be used to store a data value.
- Variable name may consist of letters, digits and the underscore(_) character,
- Examples of valid variable names are: john , Value , T_raise. Etc..
- Invalid Examples are 123, 25th, (area) etc..

Variables

- **Rules:**

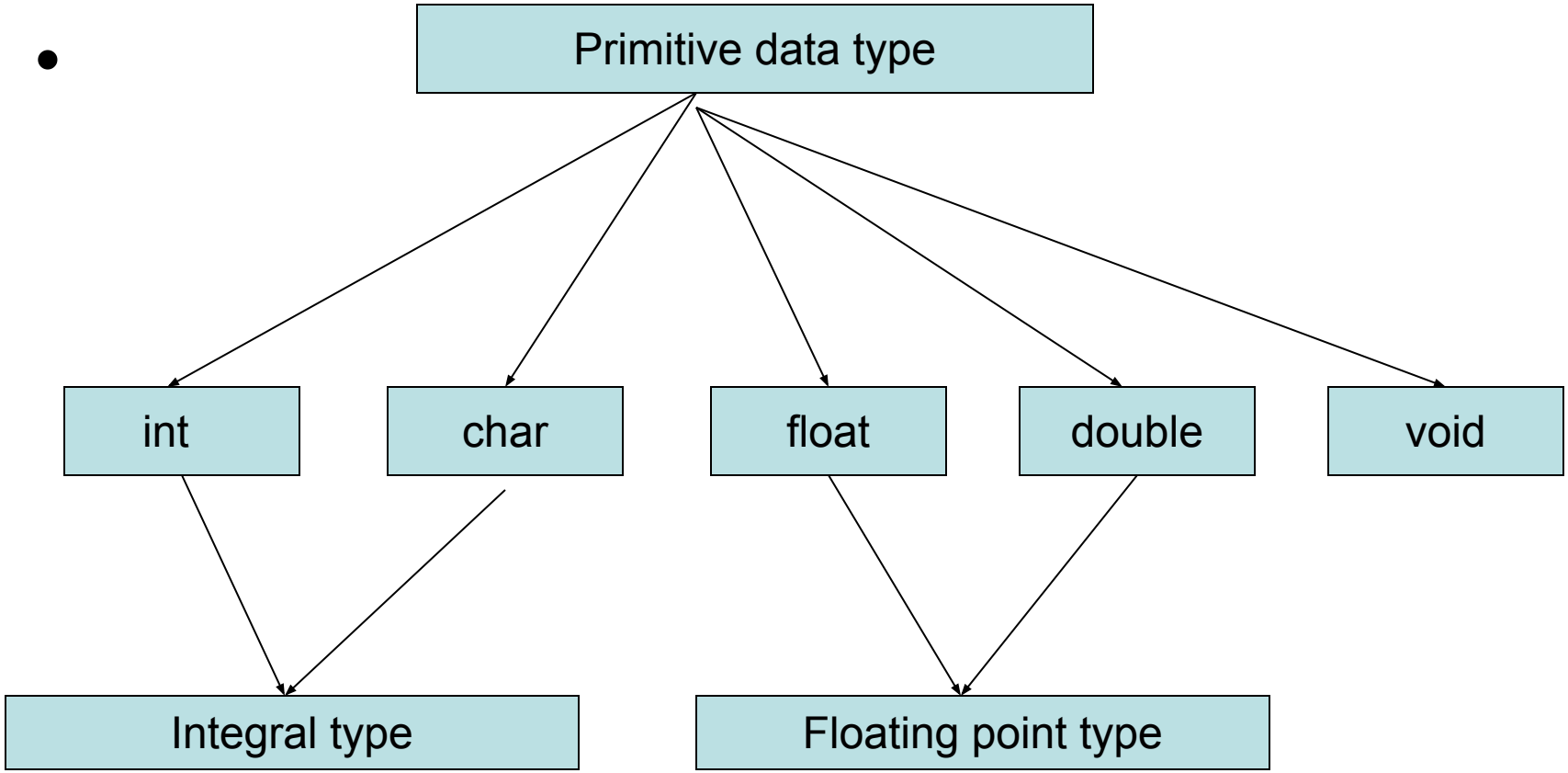
- **They must begin with a letter, some systems permit underscore as the first character.**
- **ANSI standard recognizes a length of 31 characters. However, length should not be normally more than eight characters, since only the first eight characters are treated as significant by many compilers.**
- **It should not be a keyword.**
- **White space is not allowed**

Data types

- Storage representations of various data
- ANSI standard -Three classes of data types

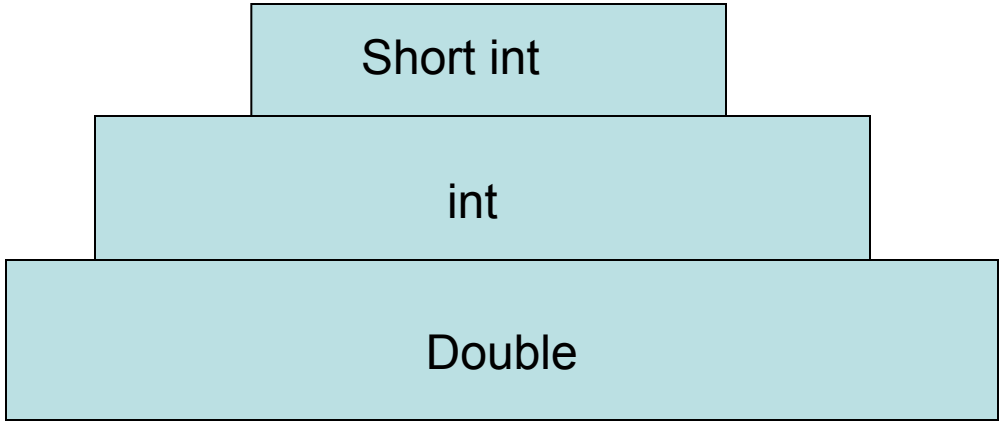
1. Primary data types
2. User-defined data types
3. Derived data types

Primitive data type

- 

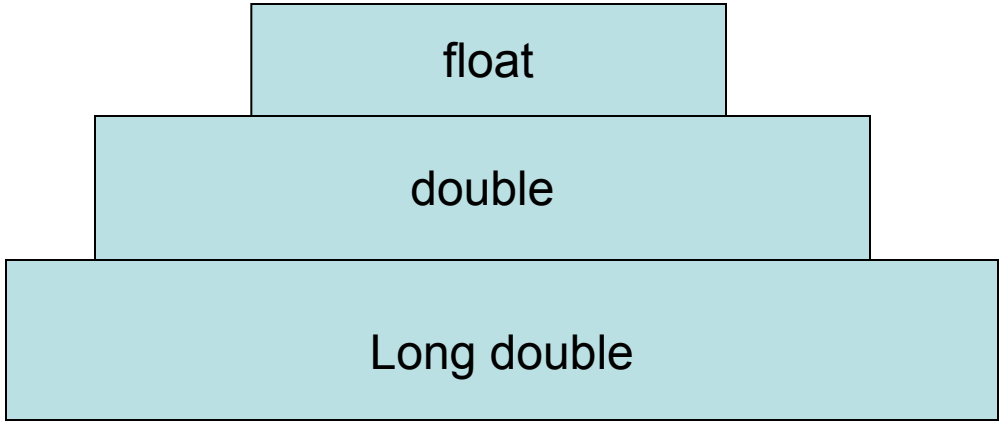
Integer Type

- **Storage size one word(16bit /32bit)**
- **Range –(-2^{15} to 2^{15} -1).**



Floating point types

- **Float-Size 32 bits**
- **6 digits precision**
- **Double –size 64 bits**
- **14 bit precision**



Character Types

- char –size 8 bits
- The qualifier signed or unsigned may be explicitly applied to char.
- While unsigned chars have values between 0 and 255.
- For signed chars Ranges from – (-2^7 to $2^7 - 1$).

Void types

- It has no values.
- This is usually used to specify the type of functions.
- It can also play the role of generic type, meaning that it can represent any of the other standard types

Declaration of variables

- It tells the compiler what the variable name is
- It specifies what type of data that variable can hold
- It allocates space according to data type
- It accepts the data of that format.
- The declaration of variables must be don’t before they are used in the program

Syntax of declaration

- **Syntax.**

  Data-type v1, v2,..vn;

-> V1, v2….,vn are the names of variables.

->Variables are seperated by commas.

-> statement must end with semicolon.

- **Example**

  int a , b, c;

  float a , b, c;

Assigning values to variables

- Variable_name = constant;
- Permits multiple assignment in one line
- Var1=cons1;
- var2=cons2;
- Var1=var2=const;
- Permits assignment at the time of declaration
- Data-type var =const; (initialization)

Defining Symbolic constants

- Unique constants in a program.
- Constants may appear repeatedly in a number of places in the program.
- Two disadvantages in the subsequent use of such problems:

1. Problem in modification of the program.
2. Problem in understanding the program.

Syntax:

#define symbolic-name value of constant.

Some valid ex.: #define PI 3.14159

Declaring a variable as constant

- We can achieve this is declaring the variable with the qualifier const at the time of initialization.
- Ex: const int class_size = 40;
- Const is a new data type qualifier.

Declaring a variable as Volatile

- The value may be changed at any time by some external sources.
- Ex: volatile int date;
- The value of date may be altered by some external factors.
- If we wish that the value must not be modified by the program while it may be altered by some other process, then we have to declare as both const and volatile
- Volatile const int location = 100

Outcomes

Students will be able to :

Define Character set, C++ Tokens, Keywords, Identifiers, Constants, Variables, Data types, Explain the Declaration of variables

ASSIGNMENT

1. What is a constant '? Explain the different types of C++ constants.
2. Explain integer constants and floating-point constants.
3. What is variable ? When do you need a variable?
4. List the rules for forming variable names.
5. How do you declare a variable in C++ ?
6. Why do you declare variables'?
7. Differentiate between the constants and the variables?

Introduction to C++ Program

- Input/output streams “iostream” **provides basic input and output services for C++ programs**.
- iostream uses the objects cin , cout , cerr , and clog for sending data to and from the standard streams input, output, error.
- iostream: iostream stands for **standard input-output stream**.

Introduction to C++ Program

- iostream stands for standard input-output stream.
- #include iostream **declares objects that control reading from and writing to the standard streams**.
- In other words, the iostream library is an object-oriented library that provides input and output functionality using streams.
- A stream is a sequence of bytes.

Introduction to C++ Program

- Namespaces are **used to organize code into logical groups and to prevent name collisions that can occur especially when your code base includes multiple libraries**.
- All identifiers at namespace scope are visible to one another without qualification.

NAMESPACE

“Using Namespace STD;”

- The std namespace in C++ contains many standard library components, including data types, functions and objects.
- When you write “using namespace std;” , you're essentially **telling the compiler to consider all the names in the std namespace as if they're in the global namespace**.

Reading & Writing Statements

- **cout** is **an object of the ostream class that is used to display output to the standard output device, usually the monitor**.
- It is associated with the standard C output stream stdout. **The insertion operator (<<)** is used with cout to insert data into the output stream.

- **cin** object in C++ is **an instance of the istream class, which is part of the standard library**.
- It receives input from the standard input device, typically the keyboard.
- **The insertion operator (>>)** is used with cout to insert data into the output stream.

A Sample C++ Program

Let's start with the short sample C++ program shown here.

```
#include <iostream>
using namespace std;

int main()
{
  int i;

  cout << "This is output.\n";  // this is a single line comment
  /* you can still use C style comments */

  // input a number using >>
  cout << "Enter a number: ";
  cin >> i;

  // now, output a number using <<
  cout << i << " squared is " << i*i << "\n";

  return 0;
}
```

Simple C++ Program

```cpp
// Your First C++ Program

#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

Output

```
Hello World!
```

Example: Program to Add Two Integers

```
#include <iostream>
using namespace std;

int main() {

  int first_number, second_number, sum;

  cout << "Enter two integers: ";
  cin >> first_number >> second_number;

  // sum of two numbers in stored in variable sumOfTwoNumbers
  sum = first_number + second_number;

  // prints sum
  cout << first_number << " + " <<  second_number << " = " << sum;

  return 0;
}
```

Output

```
Enter two integers: 4
5
4 + 5 = 9
```

C++Program to find the area of circle

```
#include <iostream>
using namespace std;
int main()
{
        float r, area;

        cout << "\n Enter Radius:";
        cin >> r;
        area = 3.14 * r * r;
        cout << "\n The Area of the circle is" << area;
}
```

Output:

Enter Radius: 6.5

The Area of the circle is 132.665

Example: Find the Factorial of a Given Number

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long factorial = 1.0;

    cout << "Enter a positive integer: ";
    cin >> n;

    if (n < 0)
        cout << "Error! Factorial of a negative number doesn't exist.";
    else {
        for(int i = 1; i <= n; ++i) {
            factorial *= i;
        }
        cout << "Factorial of " << n << " = " << factorial;
    }

    return 0;
}
```

Output

```
Enter a positive integer: 4
Factorial of 4 = 24
```

C++ LAB SYLLABUS PROGRAM

Laboratory Component Implement the following programs using cc/gcc compiler	
1	Implement the following requirement: An electricity board charges the following rates to domestic users to discourage large conceptions of energy. 0 - 100 units : Rs 1.50 per unit 101 - 200 units : Rs 1.80 per unit Beyond 200 units: Rs 2.50 per unit All users are charged a minimum of Rs 50. If the total amount is more than Rs 300 then an additional surcharge of 15% is added. The C++ program must read the names of users, number of units consumed and display the calculated charges.

C++ Classes
&
Object Oriented Programming

Structures and Classes in C++

- C++ supports all the features of structures as defined in C.
- In C++, a structure can have both variables and functions as members.
- It can declare some of its members as ‘private’.
- In C++, the structure names are stand-alone and can be used like any other type names.
- **student A; // C++ declaration**
- By default the members of a class are private, while, by default, the members of a structure are public.

Object Oriented Programming

- Object-oriented programming (OOP)
  - Encapsulates data (attributes) and functions (behavior) into packages called classes.
- So, Classes are user-defined (programmer-defined) types.
  - Data (data members)
  - Functions (member functions or methods)
- In other words, they are structures + functions

Classes in C++

- A class definition begins with the keyword *class*.
- The body of the class is contained within a set of braces, { } ; (notice the semi-colon).
- Class class_name

```
{
        .............
         ..............
         .............
 };
```

Classes in C++

- Within the body, the keywords *private:* and *public:* specify the access level of the members of the class.
  - The default is private.
- Usually, the data members of a class are declared in the *private:* section of the class and the member functions are in *public:* section.

- Class class_name

```
{
        private:        ............
                        ...............
                          .............
         public:        ...............
                          .............
};
```

Classes in C++

- <u>Member access specifiers</u>
  - public:
    - can be accessed outside the class directly.
      - The public stuff is *the interface*.
  - private:
    - Accessible only to member functions of class
    - Private members and methods are for internal use only.

- This class example shows how we can encapsulate (gather) a circle information into one package (unit or class)

```
class Circle
{
    private:
   double radius;
    public:
   void setRadius(double r);
      double  getDiameter();
   double getArea();
   double getCircumference();
};
```

Creating an object of a Class

- Declaring a variable of a class type creates an object. You can have many variables of the same type (class).
  - *Instantiation*
- Once an object of a certain class is instantiated, a new memory location is created for it to store its data members and code
- You can instantiate many objects from a class type.
  - Ex) Circle c; Circle *c;

Special Member Functions

- Constructor:
  - Public function member
  - called when a new object is created (instantiated).
  - Initialize data members.
  - Same name as class
  - No return type
  - Several constructors
  - Function overloading

Special Member Functions

```
class Circle
{
    private:
   double radius;
    public:
   Circle();
   Circle(int r);
    void setRadius(double r);
   double getDiameter();
   double getArea();
   double getCircumference();
};
```

Special Member Functions

- Class implementation: writing the code of class methods.

- There are two ways:
  - Member functions defined outside class
    - Using Binary scope resolution operator (`::`)
    - “Ties” member name to class name
    - Uniquely identify functions of particular class
    - Different classes can have member functions with same name
  - Format for defining member functions

```
ReturnType ClassName::MemberFunctionName()
{
    …
}
```

- Member functions defined inside class
  - Do not need scope resolution operator, class name

```
class Circle
{
    private:
   double radius;
    public:
   Circle() { radius = 0.0;}
   Circle(int r);
   void setRadius(double r){radius = r;}
   double getDiameter(){ return radius *2;}
   double getArea();
   double getCircumference();
};
```

```
class Circle
{
    private:
      double radius;
    public:
      Circle() { radius = 0.0;}
      Circle(int r);
      void setRadius(double r){radius = r;}
      double getDiameter(){ return radius *2;}
      double getArea();
      double getCircumference();
};
Circle::Circle(int r)
{
    radius = r;
}
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius  * (22.0/7);
}
```

Defined outside class

Accessing Class Members

- Operators to access class members
  - Identical to those for **`struct`**s
  - Dot member selection operator (`.`)
    - Object
    - Reference to object
  - Arrow member selection operator (`->`)
    - Pointers

```
class Circle
{
    private:
      double radius;
    public:
      Circle() { radius = 0.0;}
      Circle(int r);
      void setRadius(double r){radius = r;}
      double getDiameter(){ return radius *2;}
      double getArea();
      double getCircumference();
};
Circle::Circle(int r)
{
    radius = r;
}
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius  * (22.0/7);
}
```

```
void main()
{
    Circle c1,c2(7);

    cout<<"The area of c1:"
        <<c1.getArea()<<"\n";

    //c1.raduis = 5;//syntax error
    c1.setRadius(5);

    cout<<"The circumference of c1:"
    << c1.getCircumference()<<"\n";

    cout<<"The Diameter of c2:"
    <<c2.getDiameter()<<"\n";
}
```

```
class Circle
{
    private:
      double radius;
    public:
      Circle() { radius = 0.0;}
      Circle(int r);
      void setRadius(double r){radius = r;}
      double getDiameter(){ return radius *2;}
      double getArea();
      double getCircumference();
};
Circle::Circle(int r)
{
    radius = r;
}
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius  * (22.0/7);
}
```

```
void main()
{
    Circle c(7);
    Circle *cp1 = &c;
    Circle *cp2 = new Circle(7);

    cout<<"The are of cp2:"
        <<cp2->getArea();

}
```

Memory Allocation for Objects

- The member functions are created and placed in the memory space only once when they are defined.
- Since all the objects belongs to that class use the same member functions, no separate space is allocated for member functions when the objects are created.
- Only space for member variables is allocated separately for each object.
- Separate memory locations for the objects are essential, because the member variables hold different data values for different objects.

Constructors & Destructors

- The code relating to initialising data members in the class be implemented in the constructor.
- A destructor will perform destroy any dynamic memory that was allocated in the constructor or at some other point in the object life, as well as releasing any system resources that might similarly have been assigned to the object during its life.

Constructors

- Same name as class
- No return type (not even void!)
- Can be overloaded
  - Same name, differ by arguments they take
  - One with no arguments is “default” constructor
  - It is called constructor because it constructs the values of data members of the class.
- A constructor allocates space for an object and ensures that it is initialised correctly.

Example

```
int main()
{
  ToD x, y;
  return 0;
}
```

```
class ToD
{
  private:
    int h, m;
    bool PM;
  public:
    ToD(){
      cout << "ToD object created!" << endl;
    }
};
```

What’s on the screen when the program runs?

- A constructor allocates space for an object and ensures that it is initialised correctly.
- A constructor that accepts no parameters is called the default constructor.
- They should be declared in the public section.
- They can have default arguments.
- Cannot refer to their addresses
- They make implicit call to the operators new.

Default Behaviour

- If no default constructor is defined then the action on creation is to allocate store for member objects;
- if default constructors are defined for these member objects then these constructors will in turn be called and the member objects will be initialised.
- When constructors are declared in the class,initialization of the class objects becomes mandatory.

Parameterized constructors

- In practice it may be necessary to initialize the various data elements of different objects with different values when they are created.This can be achieved by passing arguments to the constructor function when the objects are created.
- The constructor that can take arguments are called parameterized constructor.

```
#include <iostream>
using namespace std;
class myclass {   int a, b;
public:
  myclass(int i, int j) {
     a=i; b=j; }
  void show() {
     cout << a << " " << b;}};
int main(){
  myclass ob(3, 5);   or myclass ob=myclass(3,5);
  ob.show();
  return 0;}
```

Constructor

- Constructor functions can also be defined as inline function.
- Parameters of a constructor can be of any type except that of the class to which it belongs.

```
class myclass { ...............
        myclass(myclass) ;
                };                                  // illegal
```

Constructor

- Constructors can accept a reference to its own class as a parameter. In such case the constructor is called a copy constructor.

```
class myclass { ...............
        myclass(myclass&) ;
        };
```

```
#include <iostream>
using namespace std;
class CRectangle {    int width, height;
  Public:      CRectangle ();
               CRectangle (int,int);
                int area (void) {return (width*height);}};
CRectangle::CRectangle () {
  Width = 5 ;height = 5; }
CRectangle::CRectangle (int a, int b) {
  width = a;  height = b; }
int main () {
  CRectangle rect (3,4);
  CRectangle rectb;
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;}
```

Constructors with default arguments

- Default arguments are formal parameters that are given a value (the default value) to use when the caller does not supply an actual parameter value. Of course, actual parameter values, when given, override the default values.

```
class X
{  public:  X(int arg1=7, int arg2=9);
....};
X::X(int arg1, int arg2)
{...
}
```

Constructors with default arguments

- The default argument constructor can be called with either one argument or no arguments.
- When called with no arguments,it becomes a default constructor.
- When both these forms are used in a class,it causes ambiguity for a statement such as
- A a; //A::A() OR A::A(int x=0)

Dynamic Initialization of objects

```
Class  xyz {
  int x,y;
  public : xyz( int a, int b) {x=a; y=b}
                xyz( ){  } };
Int main()
{  xyz obj1,obj2;
 cout <<“enter the value of x and y for object1”;
     cin>>a>>b;
 obj1= xyz(a,b);
```

Copy constructor

```
Class Point
{
  public:
  double x, y;
  Point() { x = y = 100; }
  Point(point &p)
  {
     x = p.x;
     y = p.y;
  }
  };
```

```
Point a;
Point b(a);  //call copy constructor
point c = a;  // call copy constructor
Point d;
d = a;   //copy constructor not called
```

Dynamic constructor

```
# include <iostream>
# include <cstring>
class str
{    char *name; int len;
      Public:  str() {
            len=0;
            name=new char[len+1];}
```

Dynamic constructor

```
str(char *s)  {
        len=strlen(s);
        name=newchar[len+1];
        strcpy(name,s); }
    void show() {
        cout<<"NAME IS:->"<<name<<endl;  }
    void join(str &a,str &b);
};
```

Dynamic constructor

```
void str::join(str &a,str &b)
{ len=a.len+b.len;
    delete name;
    name=newchar[len+1];
    strcpy(name,a.name);
    strcat(name,b.name);  }
main()
{    clrscr();
    char *first="HARISH";
    str n1(first),n2(“ANAND"),n3("PRATIK"),n4,n5;
```

Dynamic constructor

```
n4.join(n1,n2);
    n5.join(n4,n3);
    n1.show();
    n2.show();
    n3.show();
    n4.show();
    n5.show();
}
```

What is a destructor?

- It is a member function which deletes an object.
- A destructor function is called automatically when the object goes out of scope:
  - (1) the function ends
  - (2) the program ends
  - (3) a block containing temporary variables ends
  - (4) a *delete* operator is called
- A destructor has:
  - (i) the same name as the class but is preceded by a tilde (~)
  - (ii) no arguments and return no values

destructor

```
class string {   private:
                 char *s;
                 int size;
                 public:
                         string(char *);    // constructor
                         ~string();         // destructor  };
string::string(char *c)
{
 size = strlen(c);
 s = new char[size+1];
 strcpy(s,c);
}
string::~string()
{  delete []s; }
```

destructor

- If destructor is not defined, the compiler generates a default destructor .
- When a class contains a pointer to memory you allocate, it is your responsibility to release the memory before the class instance is destroyed.

Implementation of Destructors

```
#include <iostream>
Using namespace std;
Int count =0;
Class alpha
{  public:   alpha()
       {  count++;
          cout<<no. of obj created”<<count;
        }
      ~alpha()
     { cout<< no. of obj destroyed”<<count;
        count--;    } };
```

```
   int main()  {
   cout<< “ enter main block”;
 alpha  A1,A2,A3,A4;
{
   cout<<“\n\n enter block 1\n”;
    alpha  A5;}
{   cout<<“enter block2\n”;
   alpha A6;
}
    cout<<“ re enter main”;
                return 0;
   }
```

Introduction

- C++ is a superset of C, which has added features to support object-oriented programming.
- C++ supports classes.

C and C++

- C programmers concentrate on writing functions.
- C++ programmers concentrate on creating their own user-defined types called classes.
- Classes in C++ are a natural evolution of the C notion of **`struct`**.

Basic concepts of OOPS

- Objects
- Classes
- Inheritance
- Data Abstraction
- Data Encapsulation
- Polymorphism
- Overloading
- Reusability

Objects:

- Object is the basic unit of object-oriented programming. Objects are identified by its unique name.
- An object represents a particular instance of a class.
- An Object is a collection of data members and associated member functions also known as methods.

Classes:

- Classes are data types based on which objects are created. Objects with similar properties and methods are grouped together to form a Class.
- A Class represent a set of individual objects. Characteristics of an object are represented in a class as **Properties**.
- The actions that can be performed by objects becomes functions of the class and is referred to as **Methods**.

Classes:

- No memory is allocated when a class is created. Memory is allocated only when an object is created, i.e., when an instance of a class is created.

Inheritance:

- Inheritance is the process of forming a new class from an existing class or *base class*.
- The base class is also known as *parent class* or *super class*, The new class that is formed is called *derived class*.
- Derived class is also known as a *child class* or *sub class.* Inheritance helps in reducing the overall code size of the program, which is an important concept in object-oriented programming.

Data Abstraction:

- Data Abstraction increases the power of programming language by creating user defined data types.
- Data Abstraction also represents the needed information in the program without presenting the details.

Data Encapsulation:

- Data Encapsulation combines data and functions into a single unit called Class.
- When using Data Encapsulation, data is not accessed directly; it is only accessible through the functions present inside the class.
- Data Encapsulation enables the important concept of data hiding possible.

Polymorphism:

- Polymorphism allows routines to use variables of different types at different times.
- An operator or function can be given different meanings or functions. Polymorphism refers to a single function or multi-functioning operator performing in different ways.

Overloading:

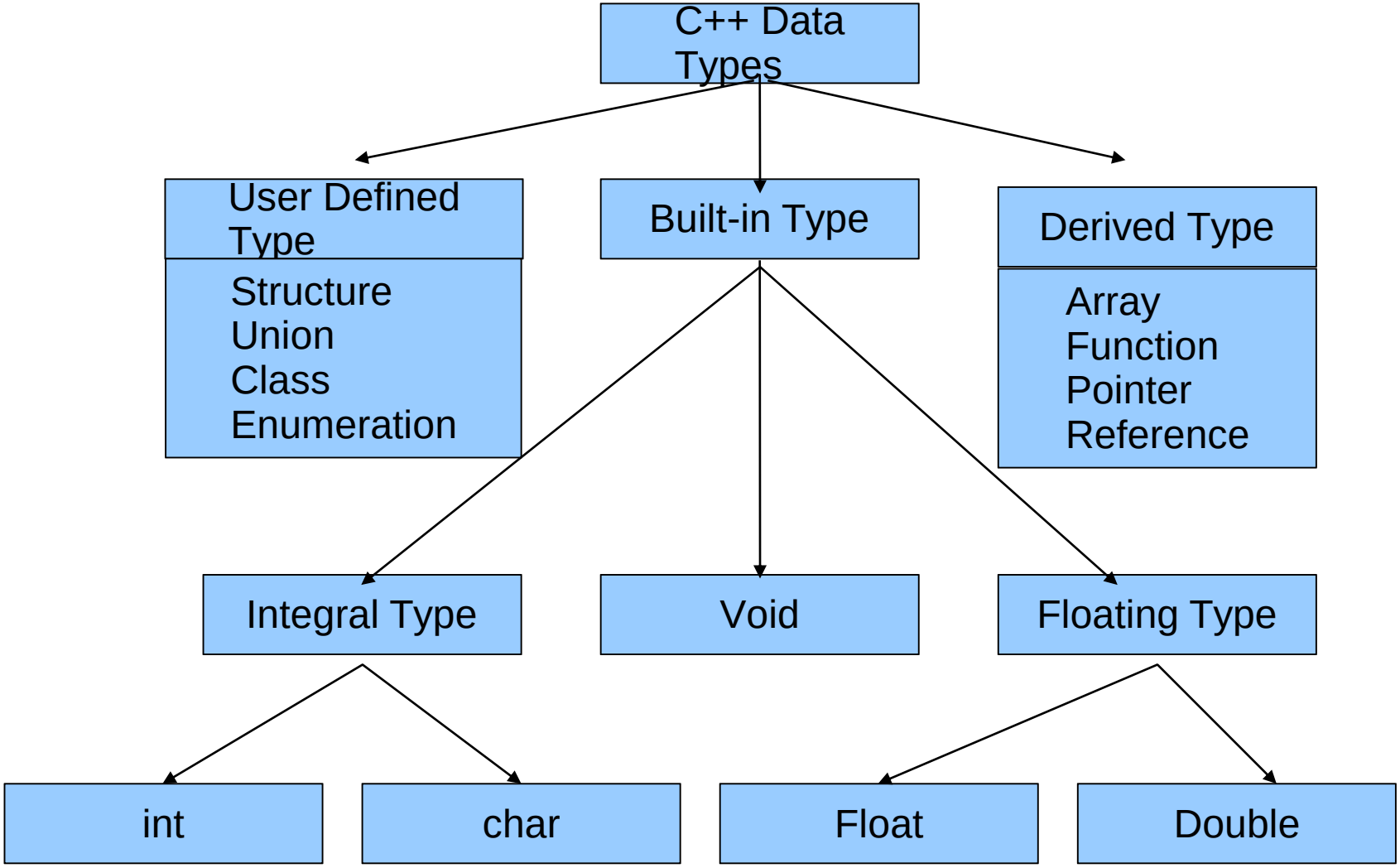
- Overloading is one type of Polymorphism. It allows an object to have different meanings, depending on its context.
- When an exiting operator or function begins to operate on new data type, or class, it is understood to be overloaded.

Reusability:

- This term refers to the ability for multiple programmers to use the same written and debugged existing class of data.
- This is a time saving device and adds code efficiency to the language. Additionally, the programmer can incorporate new features to the existing class, further developing the application and allowing users to achieve increased performance.
- This time saving feature optimizes code, helps in gaining secured applications and facilitates easier maintenance on the application.

- C++ introduces a new kind of variables known as reference variables.
- Provides aliases for previously defined variables.
- **Syntax: Data type & reference_variable= variable;**
- **EX:int total;**
- **int & sum = total;**
- Both variables refer to the same object in the memory.

Basic Data Types



VOID TYPE

- Two normal uses of void:

1)To specify the return type

2)To specify the empty parameter list

- Another interesting use is in the declaration of generic pointers
  -
- A generic pointer can be assigned a pointer value of any basic data type.
  -
  -
- Dereferencing a pointer to a void value is illegal
  -

VOID TYPE

- Assigning a void pointer to a non-void pointer without using a cast to non-void pointer type.This is not allowed in c++
- void *p1;
- char *p2;
- p2 = p1;

- P2 = (char *)p1;

User-Defined Data Types

- **Enumeration Type** :An enumeration type is a user defined type which provides a way for attaching names to numbers,thereby increasing readability of the code.
- The enum keyword automatically enumerates a list of words by assigning them values 0,1,2 and so on.
- This facility provides a alternative means for creating symbolic constants .
- **EX:enum colour {red,blue,green,yellow};**
- This DT differs slightly in c++ when compared with c.
- *The tag names can be used to declare new variables

User-Defined Data Types

Enumeration Type

Ex: colour background;

* C defines the types of enums to be imts. In c++, Each enumerated DT retains its own separate type.

EX: colour background =blue;

Colour background =8;

Colour background =(colour) 8;

- An enumerated value can be used in place of an int value.
- EX: int a= blue;

User-Defined Data Types
Enumeration Type

- C++ allows creation of anonymous enums
- EX: enum{off , on};
- These constants can be referenced in the same manner as regular constants.
- EX: int switch_1 = off;

  » int switch_2 = on;
- Practically ,enumeration is used to define symbolic constants for a switch statement.

Reference variables

- C++ introduces a new kind of variables known as reference variables.
- Provides aliases for previously defined variables.
- **Syntax: Data type & reference_variable= variable;**
- **EX:int total;**
- **int & sum = total;**
- Both variables refer to the same object in the memory.

Reference variables

- A reference variable must be initialized at the time of declaration.
- Initialization of a reference variable is completely different from assignment .
- **EX:**
- **1)int a[10];** **2) int x;**
- **int & x = a[10];** **int *p = &x;**
- **char & y = '\n';** **int & y= *p;**
-
- **3) int & q = 32;**

Reference variables

- A major application of reference variables is in passing arguments to functions.
- **EX: void fun(int & m)**
- **{ m=m++; }**
- **main( )**
- **{ int x=10;**
- **fun(x); }**
- Call by reference is used in oop because it permits the manipulation of objects by reference ,& eliminates the copying of object parameters back and forth.
- References can be created for user defined DT's also

Operators in c++

- C++ has rich set of operators. All c operators are valid in c++ also. In addition ,c++ introduces some new operators .
- **: :** scope resolution operator
- **: :*** pointer to member declarator
- **->*** pointer to member operator
- **.*** ----------,,----------
- **Delete** memory release operator
- **Endl** l line feed operator
- **New** memory allocation operator
- **Setw** field width operator

: : scope resolution operator

- Like C ,C++ is also a block -structured language
- Blocks and scopes can be used in constructing programs.
- The scope of the variable extends from the point of its declaration till the end of the block containing the declaration.
- Blocks in C++ are often nested.
- In c, the global version of variable cannot be accessed from within the inner block.c++ resolves this problem by introducing a new operator **::**
- **Syntax :: variable name**

: : scope resolution operator

```
#include <iostream>
using namespace std;
int a=10;
int main( )
{   int a=20;
        {
            int b=a;
             int a =30;
            cout<<b<<a <<:: a<<endl;
        }
      cout <<a<<:: a<<endl;
      return(0);
}
```

: : scope resolution operator

- A major application of scope resolution operator is in classes to identify class to which a member functions belongs.

Memory Management operators

- In programming there may be scenarios where programmers may not know the memory needed until run time. In this case, the programmer can opt to reserve as much memory as possible
- Assigning the maximum memory space needed to tackle this situation. This would result in wastage of unused memory spaces.
- Memory management operators are used to handle this situation in C++ programming language

Memory Management operators

- There are two types of memory management operators in C++:

  * new
  * delete

- These two memory management operators are used for allocating and freeing memory blocks in efficient and convenient ways.

- **The general syntax :**
- **pointer variable = new datatype;**
- **example:**
- **int *a=new int;**
- Dynamic variables are never initialized by the compiler. Therefore, the programmer should make it a practice to first assign them a value.

- **int *a = new int;**
- ***a = 20;**
- **or**
- **int *a = new int(20);**
- **Syntax:**
- **pt_var =new dT[10]; //for array**

delete operator:

- The delete operator in C++ is used for releasing memory space when the object is no longer needed. Once a new operator is used, it is efficient to use the corresponding delete operator for release of memory.
- **syntax:**
- **delete pointer variable;**

delete [size] pointer variable;

- **Ex :delete p;**
- **delete [ ] p;**

Delete operator

Some of the important points :

- * The programmer must take care not to free or delete a pointer variable that has already been deleted.
- * Overloading of new and delete operator is possible .*The programmer must take care not to free or delete pointer variables that have not been allocated using a new operator.

* Null pointer is returned by the new operator when there is insufficient memory available for allocation.